

Subject: How to Fine-Tune LLM

Date: from April 15, 2026 to May 7, 2026

Contents

1	Introduction	1
2	Practice: Fine-Tune Llama-3.3 using Unsloth on Single GPU	3
2.1	Environment Preparation	3
2.2	Model Loading	5
2.3	LoRA Adapter	5
2.4	Data Preparation	6
2.5	SFT Trainer	8
2.6	Model Saving and Inference	9
2.7	Evaluate on Benchmarks	10
2.8	Final Fine Tuning Results	12

How to Fine-Tune LLM

1 Introduction

Final Fine Tuning Results



Table 1: Final Fine Tuning Results on Some Benchmarks

MMLU	Hellaswag	Arc.easy	Ceval-valid
0.5334 $\pm$ 0.0040	0.5056 $\pm$ 0.0050	0.7058 $\pm$ 0.0094	0.4183 $\pm$ 0.0133

Introduction

对于一个 base model, 我们可以利用其上下文学习 (In Context Learning, ICL) 能力进行 Prompt Engineering 例如 One/Few-shot, 或者检索增强生成 (Retrieval-Augmented Generation, RAG) 提高模型的输出质量. 除此之外, 另一种提高模型输出质量的方式是微调 (Fine Tuning, FT), 其原理是: 利用某些特定领域或特殊目的构造的数据集进行有监督学习, 使得模型充分挖掘某方面的能力或某领域的知识, 从而提高相应能力或领域的输出质量, 例如文本总结、代码生成等.

与 ICL 和 RAG 等通过引入额外样例或知识提高 inference 质量不同的是, FT 将这些额外知识直接引入训练中从而直接改变模型权重. 一个较完整的 FT pipeline 如下:

1. 项目前期准备: 明确任务及目标 $\rightarrow$ 根据任务、需求、模型结构和大小选择基础模型
2. 数据准备: 数据收集+ 预处理 (labeled dataset, supervised learning)
3. 微调: 使用 supervised learning / parameter-efficient approaches (LoRA, etc.) 微调 LLM
4. 模型评估: 使用特定任务指标评估微调后的 LLM
5. 下游任务应用: 将微调好的模型应用于下游任务, 例如聊天机器人等

Finn-tuning Methods

FT 有多种方式, 例如: Instruction fine-tuning、Parameter-efficient fine-tuning (PEFT) 等.

Full Parameter Fine-Tuning (FPFT)

全量微调 (Full Parameter Fine-Tuning, FPFT) 是微调的最原始的想法, 即使用特定领域的训练数据更新模型的全部参数, 这种方式在理论上能达到的效果最高、能够最大限度地改变模型的行为, 但是算力成本极高并且容易发生灾难性遗忘^a (Catastrophic Forgetting).

^a使用新任务微调后模型在原本任务上的能力下降. 深层次原因在于不同任务的特征分布不同, FPFT 对模型的改变过大使得模型过度偏向新任务. 解决方式有参数约束 (Elastic Weight Consolidation)、数据混合 (Data Mixing)、参数高效微调 (LoRA)、架构隔离 (MoE) 等.

Instruction Fine-Tuning (IFT)

指令微调 (Instruction Fine-Tuning, IFT) 更侧重于微调数据集的构建. 在 IFT 中, 训练数据由 Instruction - Input Query - Output Response 三部分组成, 例如:

Instruction: "Translate the following English sentence to French."

Input: "The weather is nice today."

Output: "Le temps est agréable aujourd'hui."

IFT 可以使得模型更精准地遵循用户的指令, 从而提高输出质量. 与领域知识微调不同的是, IFT 的训练集无需是某个特定领域的知识, 其目的仅是教会模型“如何更好组织回答”.

Parameter-Efficient fine-tuning (PEFT)

参数高效微调 (Parameter-efficient fine-tuning, PEFT) 只更新模型的特定部分例如某些层以达到大幅减少内存占用和计算量的目的, 常用方法包括 LoRA、QLoRA、Galore 等.

LoRA (Low-Rank Adaptation) [1]

将模型的更新建模为 $W_{t+1} = W_t + \Delta W$, $W \in \mathbb{R}^{d \times k}$, 其中 ΔW 为权重更新. LoRA 的核心在于假设 ΔW 为一个低秩矩阵, 可以分解为 $\Delta W = B \times A$, 其中 $A \in \mathbb{R}^{d \times r}$, $B \in \mathbb{R}^{r \times k}$, $r \ll d$. 因此模型的更新可以通过计算两个低秩矩阵 A, B 完成.

QLoRA (Quantized Low-Rank Adaptation) [2]

QLoRA 通过将预训练权重以 NF4 (4-bit) 格式保存在显存中减小内存占用, 而在需要计算时 (例如 $A \times B$) 通过反量化 (De-quantize) 为 BF16 (16-bit) 保持计算精度, 计算完成后 16-bit 的临时权重被丢弃, 显存中继续保留 4-bit 的压缩权重.

Galore (Gradient Low-Rank Projection) [3]

Galore 认为假设权重更新是低秩的容易造成搜索空间过小而限制 FT 性能, 其核心假设是认为训练过程中梯度矩阵 G 具有低秩结构. 其将模型的更新结合优化过程, 建模为 $W_{t+1} = W_t + \eta \tilde{G}_t$, 其中 $\tilde{G}_t = P^T G_t Q$ 是原梯度矩阵的低秩投影. 在微调过程中通过不断更新投影矩阵 P, Q 可以达到更换搜索空间的目的, 从而整体上增大 FT 的搜索空间.

Reinforcement Learning Based Methods

Reinforcement Learning with Human Feedback (RLHF) [4, 5]

RHLF 暂略.

Direct Preference Optimization (DPO) [6]

DPO 暂略.

Odds Ratio Preference Optimization (ORPO) [7]

RHLF 暂略.

2 Practice: Fine-Tune Llama-3.3 using Unsloth on Single GPU

Introduction

微调 (Fine-tuning) 本质上是给预训练模型喂一些特定领域的数据集, 让它学会特定任务的过程. 此处我们选择的模型是 Llama-3.3 70B, 数据选用 `alpaca - cleaned` 数据集. 此处我们选择的微调框架为 Unsloth, 其通过“重写” Hugging Face 和 PyTorch 底层的数学运算, 让单卡训练变得更轻、更快. 整个微调的流程如下:

1. 环境准备 (Environment Preparation)
2. 模型加载 (Model Loading)
3. 加载 LoRA 适配器 (LoRA Adapter)
4. 数据预处理 (Data Preparation)
5. 配置训练参数 (SFT Trainer)
6. 执行训练与监控 (Training & Metrics)
7. 推理与保存 (Inference & Saving)
8. 测评 (Evaluate on Benchmarks)

2.1 Environment Preparation

Environment Preparation

此次微调使用的框架是 PyTorch. 根据 PyTorch 教程, 对于 CUDA 12.8 使用如下命令安装:

```
1 pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu128
```

而 CUDA 13.0 使用如下命令安装:

```
1 pip3 install torch torchvision
```

根据 Unsloth 官方文档，其已经支持 RTX 50 系列 (5060–5090), RTX PRO 6000 等，安装仅需：

```
1 pip install unsloth
```

但是若报错，可以单独创建隔离环境安装：

```
1 python -m venv unsloth
2 source unsloth/bin/activate
3 pip install unsloth
```

安装完成后进行测试：

```
1 import torch
2 from unsloth import FastLanguageModel
3 print(f"CUDA Version: {torch.version.cuda}")
4 print(f"GPU: {torch.cuda.get_device_name(0)}")
```

Output

```
CUDA Version: 12.8
GPU: NVIDIA RTX 5880 Ada Generation
```

接着安装其他需要的 packages：

```
1 pip install -r requirements.txt
```

requirements.txt

```
# Core Frameworks transformers>=4.47.0
accelerate==1.2.0
peft
bitsandbytes>=0.45.3

# Unsloth Framework
unsloth_zoo # Unsloth framework for fine tune Llama-3
trl>=0.9.0
datasets
sentencepiece
protobuf

# Recording and Tracking
weave==0.51.24
wandb

# Attention Operator
xformers>=0.0.29
```

需要注意的是此教程更适合用单卡 GPU 进行训练，多卡时在后续 SFTTrainer 中会报错，因此需要在最开始引入环境时就定义好单卡模式：

```
1 import os
2 os.environ["CUDA_VISIBLE_DEVICES"] = "3" # Only use GPU 3 for training
3 os.environ["WANDB_DISABLED"] = "true" # Disable Weights & Biases logging for
  cleaner output
```

接着引入其他 package 即可：

```

1 from unsloth import FastLanguageModel
2 import torch
3 from datasets import load_dataset
4 from trl import SFTTrainer
5 from transformers import TrainingArguments, TextStreamer
6 from unsloth import is_bfloat16_supported

```

2.2 Model Loading

Model Loading

在加载模型前，还需要配置模型的上下文窗口大小、并决定所加载的模型参数和后续微调使用的数值格式。其中 `dtype = None` 让 Unsloth 根据硬件自动决定使用 BF16 / FP16^a。此外，由于完整的 16-bit Precision Llama-3.2-70B 模型需要约 140GB 显存 (VRAM)，因此为减少显存占用需要加载 4-bit 量化模型，此时只需约 40GB VRAM (此处选用 6-8 GB VRAM 的 Llama-3.2-3B):

```

1 # 1. Basic Configurations
2 max_seq_length = 2048      # Supports RoPE Scaling internally
3 dtype           = None     # None for auto detection (Float16/BFloat16)
4 load_in_4bit    = True     # Use 4-bit quantization to reduce VRAM usage

```

确定好配置之后，需要从 Unsloth 的 `FastLanguageModel` 中加载模型和对应的 `Tokenizer`:

```

1 # 2. Load Model and Tokenizer
2 model, tokenizer = FastLanguageModel.from_pretrained(
3     model_name = "unsloth/Llama-3.2-3B-Instruct-bnb-4bit", # Start with 3B for
4     # model_name = "unsloth/Llama-3.2-70B-Instruct-bnb-4bit", # Switch to 70B for
5     # actual fine-tuning
6     max_seq_length = max_seq_length,
7     dtype = dtype,
8     load_in_4bit = load_in_4bit,
9 )

```

```

1 ...
2 Unsloth: Fast downloading is enabled - ignore downloading bars which are red
3 colored!
4 Loading weights: 100%||||||| 254/254 [00:00<00:00, 332.82it/s]
5 Unsloth: Will load unsloth/Llama-3.2-3B-Instruct-bnb-4bit as a legacy tokenizer.

```

^aBF16 和 FP16 都使用 16 bits 加速训练，但 BF16 有着更大的动态范围 (dynamic range) (与 FP32 相似)

2.3 LoRA Adapter

LoRA Adapter

为高效部署，我们选用 PEFT 中的 LoRA。我们主要需要控制 ① 秩 (Rank): 控制 $\Delta W = B \times A, A \in \mathbb{R}^{d \times r}, B \in \mathbb{R}^{r \times k}, r \ll d$ 中 r 的大小。② 微调目标模块 (target modules): 控制 LoRA 作用的区域，例如 `q_proj, k_proj, v_proj, o_proj` 控制 Attention (注意力机制)，主要影响模型如何理解上下文；`gate_proj, up_proj, down_proj` 控制 MLP，主要影响模型的知识存储和推理。③ 缩放系数 (lora_alpha): 控制模型通过 LoRA 改变的程度，即 $\Delta W = \frac{\alpha}{r} BA$ ，较低的 α 会使模型更多地保留原来的能力。④ 梯度检查点 (Gradient Checkpointing): 全部保存前向传播中每一层的激活值 (Activations) 需要大量内存，为节省内存可以只保存一部分例如每 4 层的值，在反向传播中若计算到未保存的层时再临时前向传播一下即可。

```

1 model = FastLanguageModel.get_peft_model(

```

```

2     model,
3     r = 16,                # Rank: controls the number of trainable parameters
4     target_modules = [    # The parts of the model we want to fine-tune
5         "q_proj", "k_proj", "v_proj", "o_proj",
6         "gate_proj", "up_proj", "down_proj",
7     ],
8     lora_alpha = 16,        # Scaling factor for LoRA updates
9     lora_dropout = 0,      # For performance, recommended to 0
10    bias = "none",          # For performance, set to none
11    use_gradient_checkpointing = "unsloth", # Extremely important: enabling this
        can significantly save GPU memory
12    random_state = 3407,
13    use_rslora = False,     # Rank-stable LoRA,
14    loftq_config = None,   # Quantization initialization configuration
15 )

```

2.4 Data Preparation

Alpaca

alpaca [8] 包含了 52,002 条由 OpenAI 的 text-davinci-003 生成的指令和演示，其每一条数据为一组 (Instruction, Input, Output, Text) pair，其中 Text 为所有内容组合而成的 prompt:

Text Example for Alpaca Dataset

Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.

Instruction:

Identify the odd one out.

Input^a:

Twitter, Instagram, Telegram

Response:

Telegram

^a实际上在 alpaca 数据集中绝大多数 pairs 都没有 input 部分。

alpaca – cleaned 是 Stanford 的 alpaca 数据集清洗后的版本，修复、删除了一些错误、幻觉等数据，共包括 51,760 组 (Instruction, Input, Output) pairs.

Data Preparation

首先使用 load_dataset 加载数据集并查看:

```

1 dataset = load_dataset("yahma/alpaca-cleaned", split="train")
2
3 raw_example = dataset[1]
4
5 print(raw_example, "\n")
6 print(f"Example type: {type(raw_example)}", "\n")
7
8 for key, value in raw_example.items():
9     print(f"{key:<15}: {value}")

```

Output

```
{'output': 'The three primary colors are red, blue, and yellow. These colors are called primary because they cannot be created by mixing other colors and all other colors can be made by combining them in various proportions. In the additive color system, used for light, the primary colors are red, green, and blue (RGB).', 'input': '', 'instruction': 'What are the three primary colors?'}
```

Example type: <class 'dict'>

output : The three primary colors are red, blue, and yellow. These colors are called primary because they cannot be created by mixing other colors and all other colors can be made by combining them in various proportions. In the additive color system, used for light, the primary colors are red, green, and blue (RGB).

input :

instruction : What are the three primary colors? ...

可以发现数据集中每一条数据实际上是一个字典 (dict), 包含 `instruction`, `input`, `output` 三组键值对, 我们希望将他们合并成一条语料 (`text`), 同时在语料最后加上 `EOS_TOKEN` 进行区分:

```
1 # Prompt Formatting
2 alpaca_prompt = """Below is an instruction that describes a task, paired with an
   input that provides further context. Write a response that appropriately
   completes the request.

3
4 ### Instruction:
5 {}

6
7 ### Input:
8 {}

9
10 ### Response:
11 {}"""
12
13 EOS_TOKEN = tokenizer.eos_token
```

在定义了模板后, 就可以定义函数将原本每条数据为 `Instruction`, `Input`, `Output` 键值对的数据变成每条仅含 `text` 的数据 (注意此时每个 `example` 仍然是 `dict` 但仅含 `text` 一个键值对):

```
1 def formatting_prompts(examples):
2     instructions = examples["instruction"]
3     inputs       = examples["input"]
4     outputs      = examples["output"]
5     texts = [alpaca_prompt.format(instruction, input, output) + EOS_TOKEN for
6               instruction, input, output in zip(instructions, inputs, outputs)]
7     return {"text": texts}
8 dataset = dataset.map(formatting_prompts, batched = True)

1 print(f"Type: {type(dataset[1])}", "\n")
2 print(dataset[1]["text"])
```

Output

Type: <class 'dict'>

Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.

Instruction:

What are the three primary colors?

Input:

Response:

The three primary colors are red, blue, and yellow. These colors are called primary because they cannot be created by mixing other colors and all other colors can be made by combining them in various proportions. In the additive color system, used for light, the primary colors are red, green, and blue (RGB).< |eot_id| >

最后进行训练集和验证集区分:

```
1 train_val = dataset.train_test_split(test_size=0.03, seed=3407)
2 train_dataset = train_val["train"]
3 eval_dataset = train_val["test"]
```

2.5 SFT Trainer

Supervised Fine-Tuning (SFT) Trainer

在训练模型时我们使用现成的 `SFTTrainer`，只需要配置好一些参数即可^a (注意此处我们使用单卡模式，已在最开始的 `Environment Preparation` 处说明)。

```
1 trainer = SFTTrainer(
2     model = model,
3     tokenizer = tokenizer,
4     train_dataset = train_dataset,
5     eval_dataset = eval_dataset,
6     dataset_text_field = "text",
7     max_seq_length = max_seq_length,
8     dataset_num_proc = 2, # Number of CPU cores used for data loading
9     packing = False,
10    args = TrainingArguments(
11        per_device_train_batch_size = 4, # Number of parallel data per GPU
12        gradient_accumulation_steps = 8, # Effective batch size =
13            per_device_train_batch_size * gradient_accumulation_steps
14        warmup_steps = 5,
15        max_steps = 60, # Total number of training steps
16        learning_rate = 2e-4,
17        fp16 = not is_bfloat16_supported(), # Use FP16 if BFloat16 is not
18            supported
19        bf16 = is_bfloat16_supported(),
20        logging_steps = 1, # Log training loss every 1 step
21        optim = "adamw_8bit",
22        weight_decay = 0.01,
23        lr_scheduler_type = "linear", # or "cosine" etc.
24        seed = 3407,
25        eval_strategy = "steps",
26        # evaluation_strategy="steps", # for older versions of transformers
27        eval_steps=20, # Evaluate every 20 steps
```

```

26     save_strategy="steps",
27     save_steps=20, # Save checkpoint every 20 steps
28     output_dir = "outputs",
29     load_best_model_at_end=True,
30     metric_for_best_model="loss",
31     greater_is_better=False, # because we want to minimize the loss
32     # report_to = "wandb", # Upload the loss curve to the cloud
33 ),
34 )

```

说明：① `per_device_train_batch_size`: 每张 GPU 上并行处理数据量。

② `gradient_accumulation_steps`: 梯度累加，模拟大 batch size。

“注意在较新版本的 `transformers` 库中使用 `eval_strategy` 而较早版本中使用 `evaluation_strategy`。

2.6 Model Saving and Inference

Train and Save Model

确定训练配置后即可进行训练与模型保存：

```

1 trainer_stats = trainer.train()
2
3 # Save Model
4 model.save_pretrained("lora_model")
5 tokenizer.save_pretrained("lora_model")

```

Inference

在模型完成微调之后，我们希望看看模型的对话能力如何，即进行 inference：

```

1 # Inference
2 FastLanguageModel.for_inference(model) # Switch to inference mode
3 # instruction = "Continue the Fibonacci sequence."
4 # input_content = "1, 1, 2, 3, 5, 8"
5 instruction = "Describe the structure of an atom."
6 input_content = ""
7 prompt = alpaca_prompt.format(instruction, input_content, "")
8 inputs = tokenizer([prompt], return_tensors = "pt").to("cuda")
9 print(f"{'='*30} PROMPT {'='*30}")
10 print(prompt)
11
12 outputs = model.generate(**inputs, max_new_tokens=256, use_cache=True,
13                          repetition_penalty=1.2, # Penalize repetition to
14                          encourage more diverse outputs
15                          temperature=0.7, # Control randomness in
16                          generation (lower is more deterministic)
17 )
18 decoded_output = tokenizer.batch_decode(outputs)
19 response = decoded_output[0].split("### Response:")[1]
20 print(f"{'='*30} RESPONSE {'='*30}")
21 print(response)

```

Output

===== RESPONSE =====

An atom consists of three main subatomic particles: protons, neutrons and electrons. Protons and neutrons are found in the nucleus or center of the atom while electrons orbit around it. The number of each type of particle determines the element's atomic number and mass.

The proton has a positive charge while neutron has no charge but contributes to the overall mass of the atom by adding to its nucleon count. Electrons have a negative charge and their movement around the nucleus gives rise to chemical reactivity of elements as they tend to form bonds with other atoms through sharing electrons' energy levels.

...

In summary, the basic structural components of an atom include protons, neutrons (in some cases), and electrons; these parts work together under forces like gravity, electromagnetism, and the strong nuclear force, to define what we know today as matter itself.< |eot_id| >

2.7 Evaluate on Benchmarks

Introduction

Benchmark (基准) 是测试模型能力的重要部分，其相当于大语言模型 (LLM) 的“标准化考试”。常见的 Benchmark 有：

1. MMLU: Massive Multitask Language Understanding, 考察综合百科知识. 包含了数学、人文等 57 个学科的选择题，难度从高中到专家级别，CMMLU 是 MMLU 的中文版。
2. HellaSwag: 考察常识推理. 会给出一个场景的开头，让模型选择最符合常理的后续发展。
3. GSM8K: 考察数学与逻辑计算. 高质量的小学级别数学应用题库，主要测试模型的基础算术和多步推理能力。
4. HumanEval: 考察代码编写能力. OpenAI 发布的测试集，给模型一段 Python 函数的注释和签名，让模型把代码写完，然后运行单元测试看是否通过。

测 Benchmark 是考察模型的重要一环，例如 DeepSeek-V2 的部分测试结果如下：

Table 2: Standard Benchmark (Models smaller than 16B)

Benchmark	Domain	DeepSeek 7B Chat (SFT)	DeepSeekMoE 16B Chat (SFT)	DeepSeek-V2-Lite 16B Chat (SFT)
MMLU	English	49.7	47.2	55.7
BBH	English	43.1	42.2	48.1
C-Eval	Chinese	44.7	40.0	60.1
CMMLU	Chinese	51.2	49.3	62.5
HumanEval	Code	45.1	45.7	57.3
MBPP	Code	39.0	46.2	45.8
GSM8K	Math	62.6	62.2	72.0
Math	Math	14.7	15.2	27.9

Evaluate on Benchmarks

测试 Benchmark 可以使用 package `lm_eval[hf]`, 安装方式如下:

```
1 pip install "lm_eval[hf]"
```

在测试时需要定义好我们测试的任务、使用的 GPU 等:

```
1 import os
2 import subprocess
3 from datetime import datetime
4
5 os.environ["CUDA_VISIBLE_DEVICES"] = "0" # use only one GPU for benchmarking
6
7 # =====
8 # 1. Basic Configurations
9 # =====
10 lora_model_path = xxx # your path here
11 base_model_path = xxx # your path here
12 output_dir = xxx      # your path here
13
14 def run_benchmarks(peft_path, base_path, output_dir):
15     print(f"[INFO] Base Model: {base_path}")
16     if not os.path.isdir(peft_path):
17         raise FileNotFoundError(f"PEFT Model not found: {peft_path}")
18     else:
19         print(f"[INFO] LoRA Weights: {peft_path}")
20
21     os.makedirs(output_dir, exist_ok=True) # make sure the output directory
22     exists, if not it will be created
23     print(f"[INFO] Test results will be saved in: {output_dir}")
24
25     # =====
26     # 2. Build Test Cases
27     # =====
28     tasks = "hellaswag,mmlu"
29
30     # Build HuggingFace model arguments, combining base + peft
31     model_args = f"pretrained={base_path},peft={peft_path},tokenizer={base_path}"
32
33     cmd = [
34         "lm-eval", "run",
35         "--model", "hf",
36         "--model_args", model_args,
37         "--tasks", tasks,
38         "--device", "cuda:0", # Because CUDA_VISIBLE_DEVICES="3" is specified
39         externally, cuda:0 maps to GPU 3
40         "--batch_size", "auto:4",
41         "--output_path", output_dir,
42         # "--limit", "100" # Limit to 100 samples per task for quick benchmarking
43     ]
44
45     # print(f"\n[INFO] Executing command:\n{' '.join(cmd)}\n")
46     print(f"\n[INFO] Running benchmark: {tasks}\n")
47
48     # =====
49     # 3. Run the Benchmark Command
50     # =====
51
52 if __name__ == "__main__":
53     run_benchmarks(lora_model_path, base_model_path, output_dir)
```

如果需要改成多卡并行处理数据, 则需要改动一部分代码:

```
1 ...
2 # delete os.environ["CUDA_VISIBLE_DEVICES"] = "0"
```

```

3  ...
4  cmd = [
5      "accelerate", "launch",
6      "--multi_gpu",
7      "--num_processes", f"{num_gpus}",
8      "-m", "lm_eval",
9
10     "--model", "hf",
11     "--model_args", model_args,
12     "--tasks", tasks,
13     "--batch_size", "auto:4",
14     "--output_path", output_dir,
15     "--apply_chat_template",
16     # "--limit", "100" # Limit to 100 samples per task for quick
        benchmarking
17 ]
18 ...

```

2.8 Final Fine Tuning Results

Final Fine Tuning Results

最终我们使用单卡 GPU 训练了 1000 steps，训练结果如下：

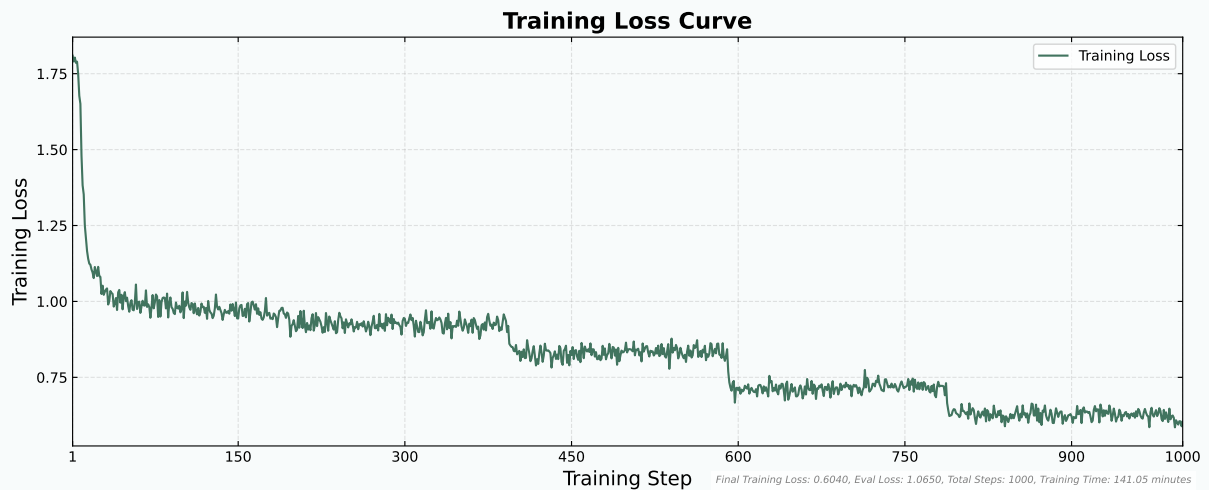


Table 3: Final Fine Tuning Results on Some Benchmarks

MMLU	Hellaswag	Arc_easy	Ceval-valid
0.5334 $\pm$ 0.0040	0.5056 $\pm$ 0.0050	0.7058 $\pm$ 0.0094	0.4183 $\pm$ 0.0133

在 Base Pretrained Models 的数据中 Llama 3.2 3B MMLU 得分为 0.58，因此可以看出已经发生了过拟合，模型的一般性能力产生下降. 事实上通过下面的 validation 曲线也可以看出：



Last updated: May 7, 2026

References

- [1] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.
- [2] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems*, 36:10088–10115, 2023.
- [3] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. Galore: Memory-efficient llm training by gradient low-rank projection. *arXiv preprint arXiv:2403.03507*, 2024.
- [4] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- [5] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- [6] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.
- [7] Jiwoo Hong, Noah Lee, and James Thorne. Orpo: Monolithic preference optimization without reference model. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11170–11189, 2024.
- [8] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.